

اصل وارونگی وابستگی (Dependency Inversion Principle یا DIP) می‌گوید:

1. ماژول‌های سطح بالا نباید به ماژول‌های سطح پایین وابسته باشند. هر دو باید به انتزاع‌ها (Interfaces یا Abstract Classes) وابسته باشند.
 2. انتزاع‌ها نباید به جزئیات وابسته باشند، بلکه جزئیات باید به انتزاع‌ها وابسته باشند.
- به عبارت ساده: کلاس‌های با منطق اصلی برنامه (سطح بالا) نباید مستقیماً به جزئیات پیاده‌سازی (سطح پایین) وابسته باشند. این کار باعث کاهش Coupling و افزایش Flexibility و Testability می‌شود.

مثال نقض اصل DIP

فرض کنید یک کلاس گزارش‌ساز داریم که مستقیماً روی یک کلاس چاپگر (Printer) کار می‌کند:

```
// کلاس سطح پایین
public class Printer
{
    public void Print(string content)
    {
        Console.WriteLine(content);
    }
}

// کلاس سطح بالا
public class Report
{
    private Printer printer = new Printer();

    public void GenerateReport()
    {
        string content = "Report Content";
        printer.Print(content);
    }
}
```

مشکل:

کلاس Report مستقیماً به کلاس Printer وابسته است. اگر بخواهیم گزارش را به جای چاپگر، در فایل ذخیره کنیم یا به ایمیل بفرستیم، باید کلاس Report را تغییر دهیم. این یعنی High-level module به Low-level module وابسته است و DIP نقض شده است.

اصلاح با استفاده از DIP

با تعریف یک انتزاع برای چاپگر:

```
// انتزاع
public interface IPrinter
{
    void Print(string content);
}

// کلاس سطح پایین پیاده‌ساز انتزاع
public class Printer : IPrinter
{
    public void Print(string content)
    {
        Console.WriteLine(content);
    }
}

// کلاس سطح بالا
public class Report
{
    private IPrinter printer;

    public Report(IPrinter printer)
    {
        this.printer = printer;
    }

    public void GenerateReport()
    {
        string content = "Report Content";
        printer.Print(content);
    }
}
```

مزیت:

حالا کلاس Report به جزئیات Printer وابسته نیست، بلکه به انتزاع IPrinter وابسته است.

می‌توانیم انواع مختلف چاپگر یا ذخیره‌سازی را بدون تغییر Report پیاده کنیم.

وابستگی کاهش یافته و کلاس‌ها انعطاف‌پذیرتر شده‌اند.